

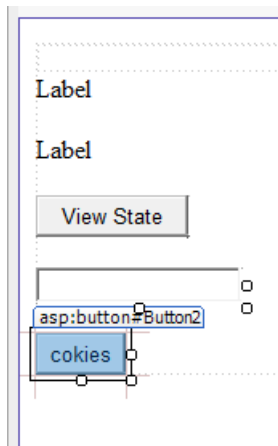
Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester IV)
Dot .Net Core and Progressive Web Application Practical

Practical – V

Roll No.: T011	Name: Harshita Kanojia
Class: TYIT	Batch: 1
Date of Assignment: 21/07/2026	Date/Time of Submission: 21/07/2026

Aim(A) :Create a web application to demonstrate the use of different types of Cookies.

Code:



WebForm1.aspx.cs :

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace WebApplication1
{
    public partial class WebForm1 : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if (IsPostBack)
            {
                int viewStateVal = Convert.ToInt32(ViewState["count"]) + 1;
                Label2.Text = viewStateVal.ToString();
            }
            else {
                ViewState["count"] = 1;
            }
        }
    }
}
```

```

protected void Button1_Click(object sender, EventArgs e)
{
    Label1.Text = ViewState["count"].ToString();
}

protected void Button2_Click(object sender, EventArgs e)
{
    HttpCookie h = new HttpCookie("name");
    h.Value = TextBox1.Text;
    Response.Cookies.Add(h);
    Response.Redirect("WebForm2.aspx");
}
}
}

```

WebForm2.aspx.cs :

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace WebApplication1
{
    public partial class WebForm2 : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if (Request.Cookies["name"] != null) {
                Response.Write("Welcome"+Request.Cookies["name"].Value);
            }
        }
    }
}

```

Output:

The screenshot displays two web forms side-by-side. The left form contains a 'View State' button and a text box. The right form contains a 'View State' button, a text box, and a 'cookies' button. Below the forms, the text 'WelcomeHarshita Kanojia' and 'Label' is shown, indicating the output of the application.

Aim(B) : Create a Product Table in SQL Server and perform Insert Operation using Entity Framework Core.

Code:

Step 1: Create Database

```

CREATE DATABASE ProductDB;
GO

```

```

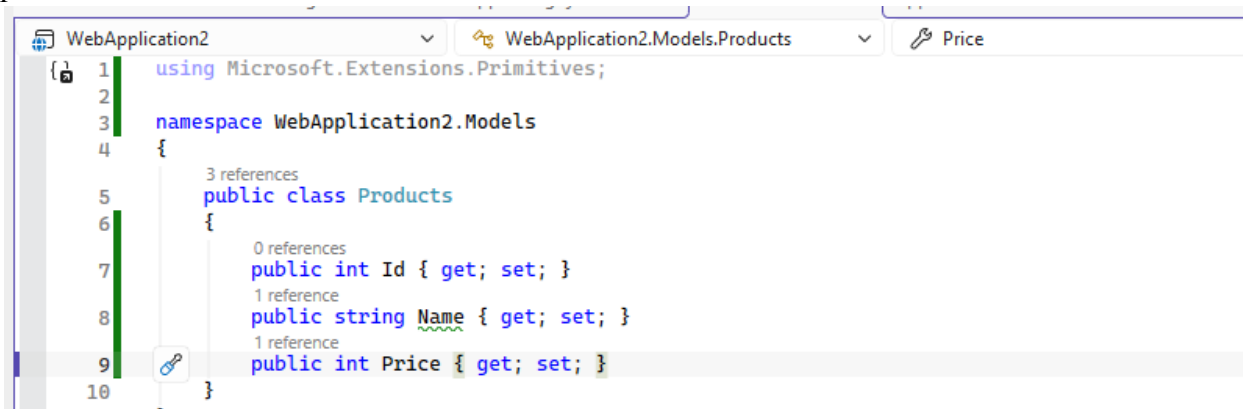
USE ProductDB;

```

No need to create the table manually. EF Core will create it.

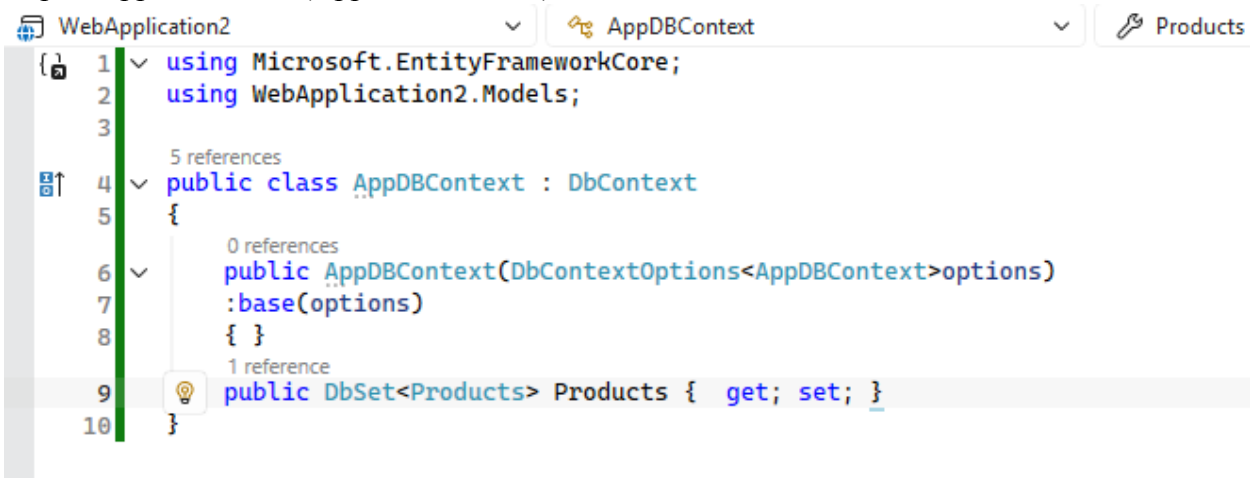
Step 2: Product Model (Product.cs)

public class Product



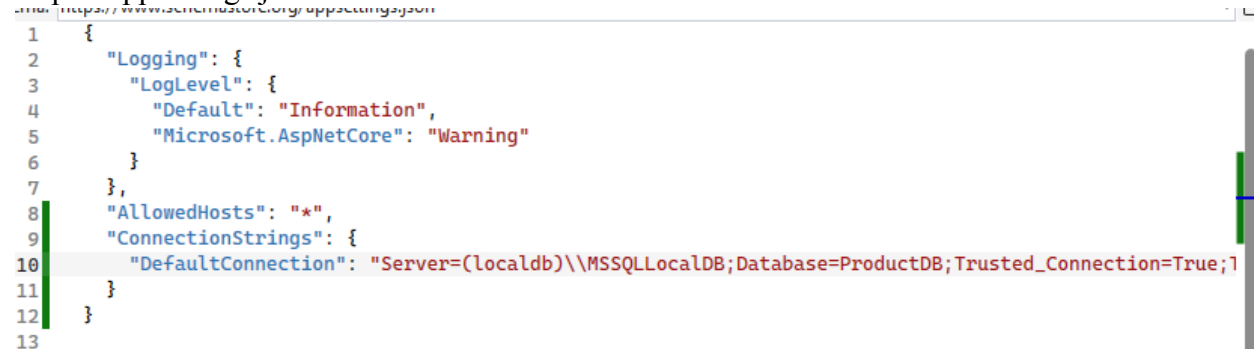
```
1 using Microsoft.Extensions.Primitives;
2
3 namespace WebApplication2.Models
4 {
5     3 references
6     public class Products
7     {
8         0 references
9         public int Id { get; set; }
10        1 reference
11        public string Name { get; set; }
12        1 reference
13        public int Price { get; set; }
14    }
15 }
```

Step 3: ApplicationDbContext (AppDbContext.cs)



```
1 using Microsoft.EntityFrameworkCore;
2 using WebApplication2.Models;
3
4 5 references
5 public class ApplicationDbContext : DbContext
6 {
7     0 references
8     public ApplicationDbContext(DbContextOptions<ApplicationDbContext>options)
9     :base(options)
10    { }
11    1 reference
12    public DbSet<Products> Products { get; set; }
13 }
```

Step 4: appsettings.json



```
1 {
2   "Logging": {
3     "LogLevel": {
4       "Default": "Information",
5       "Microsoft.AspNetCore": "Warning"
6     }
7   },
8   "AllowedHosts": "*",
9   "ConnectionStrings": {
10    "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=ProductDB;Trusted_Connection=True;"
11  }
12 }
13 }
```

Step 5: Program.cs

```

Application2  Program  Stop-Server-Start
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

app.UseStaticFiles();

app.UseRouting();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();

```

Step 6: HomeController

```

Application2  HomeController  Index()
1 using Microsoft.AspNetCore.Mvc;
2 using WebApplication2.Models;
3
4 1 reference
5 public class HomeController : Controller
6 {
7     private readonly AppDbContext _context;
8
9     0 references
10    public HomeController(AppDbContext context)
11    {
12        _context = context;
13    }
14
15    0 references
16    public IActionResult Index()
17    {
18        Products p = new Products();
19
20        p.Name = "Laptop Harshita";
21        p.Price = 50000;
22
23        _context.Products.Add(p);
24        _context.SaveChanges();
25
26        return Content("Product Inserted Successfully");
27    }
28 }

```

Step 7: Install Packages

Open Package Manager Console

Install-Package Microsoft.EntityFrameworkCore.SqlServer

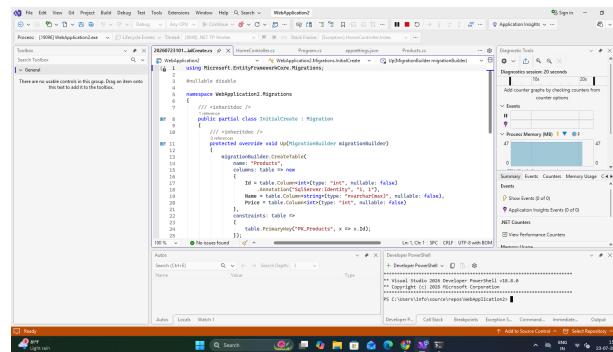
Install-Package Microsoft.EntityFrameworkCore.Tools

Step 8: Migration
Add-Migration InitialCreate
Update-Database

Step 9: Run
Run the project and open:
<https://localhost:xxxx/Home/Index>
Output:
Product Inserted Successfully

Step 10: Verify
Open SQL Server and run:
SELECT * FROM Products;

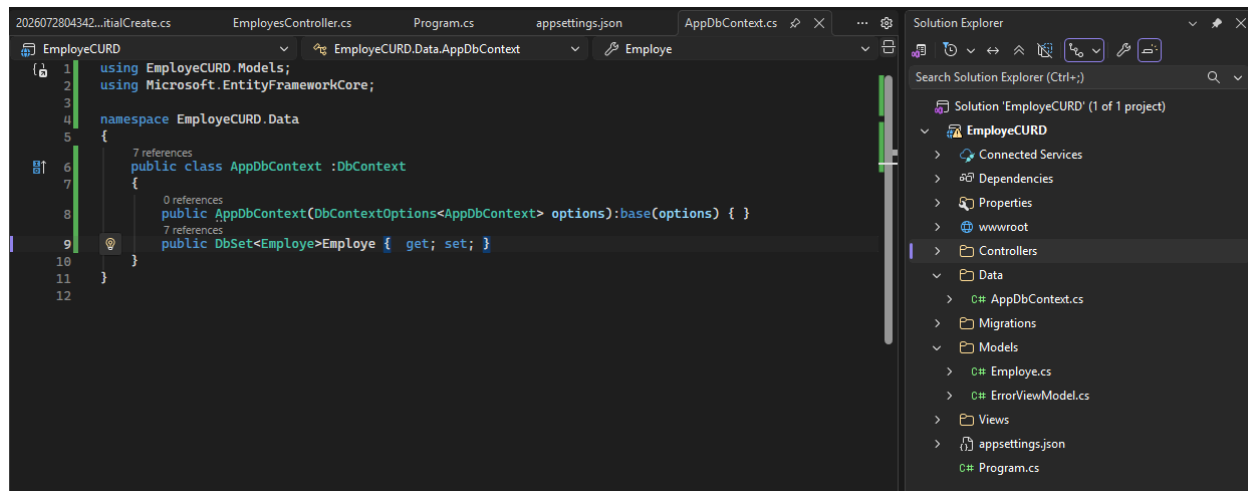
Output:

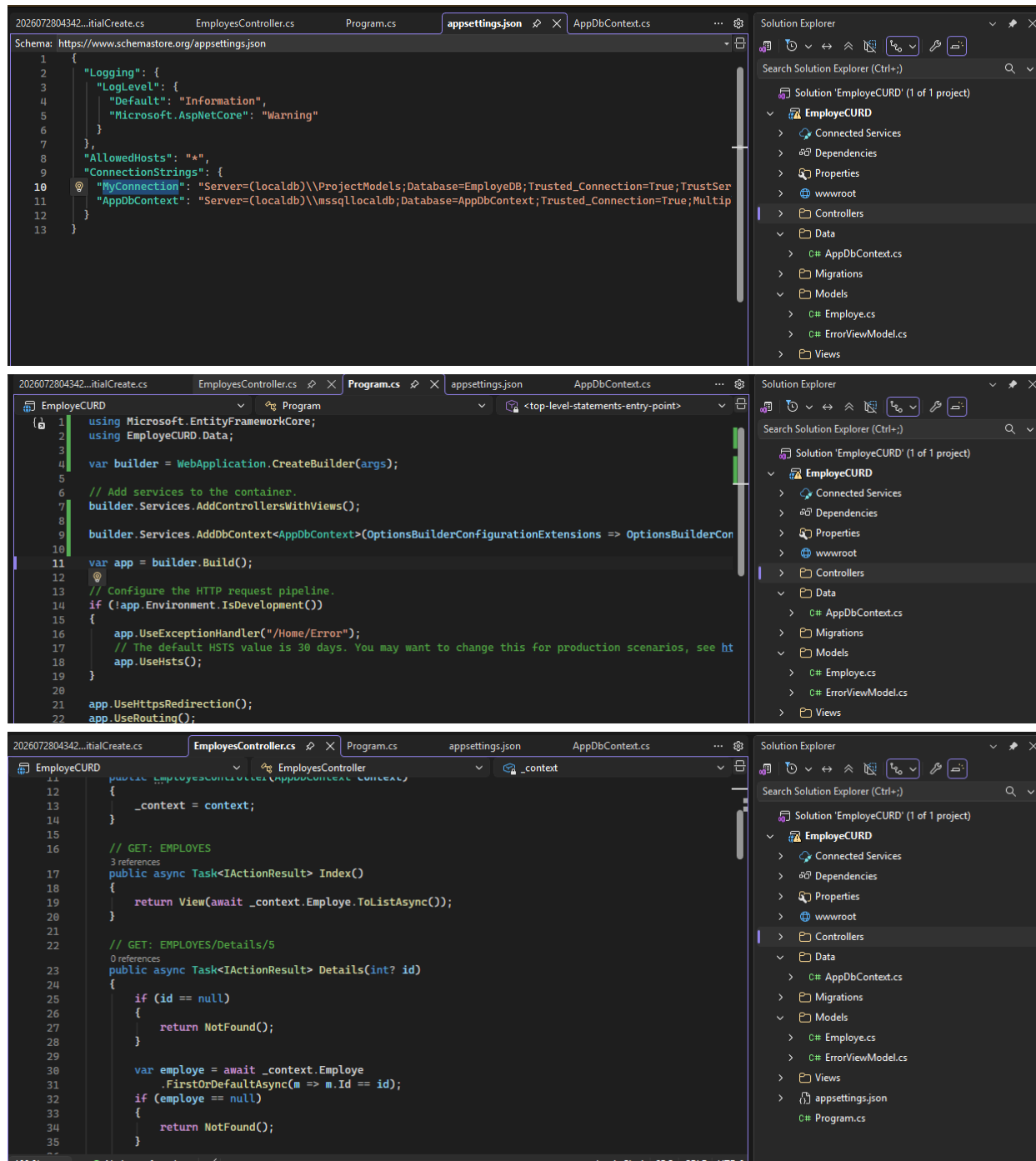


Product Inserted Successfully

Aim(C) :Develop a web application to perform CRUD Operations (Create, Read, Update, Delete) on the Employee table using EF Core.

CODE :



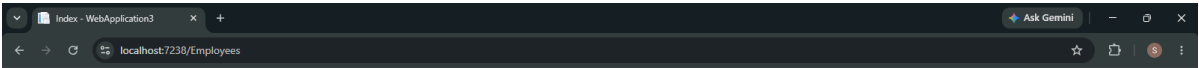


OUTPUT :



Welcome

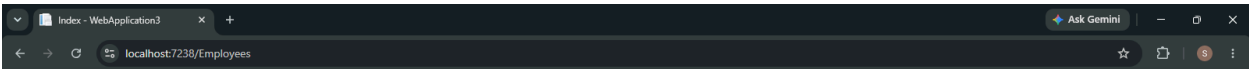
Learn about [building Web apps with ASP.NET Core](#).



Index

[Create New](#)

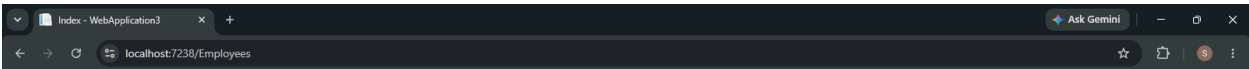
Name	Department	Salary
------	------------	--------



Index

[Create New](#)

Name	Department	Salary	
harshita	TYIT	7826378	Edit Details Delete



Index

[Create New](#)

Name	Department	Salary	
sachin	TYIT	7287828	Edit Details Delete

Aim(D) :Authentication and Authorization in ASP.NET Core. ye code hai tho please share kar de Steps :-

1. Register Auth Services: In Program.cs, add services using `AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme).AddCookie(...)` and `AddAuthorization()`.
2. Configure Pipeline: Add `app.UseAuthentication()` before `app.UseAuthorization()` in Program.cs.
3. Build Login Logic: In HomeController.cs, validate user credentials, create a ClaimsIdentity, and call `await HttpContext.SignInAsync(...)`.
4. Protect Route: Add the `[Authorize]` attribute directly above the `Secure()` action method.
5. Build View & Logout: Create Views/Home/Index.cshtml with a login form, and implement `await HttpContext.SignOutAsync(...)` inside the `Logout()` action.
1. Create an ASP.NET Core MVC project.
2. Enable Cookie Authentication in Program.cs.
3. Create Login action in HomeController.
4. Verify username and password.
5. Redirect to Secure page after successful login.
6. Use `[Authorize]` to protect the Secure page.
7. Logout and redirect to Login.

Program.cs

```
using Microsoft.AspNetCore.Authentication.Cookies;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Home/Index";
    });

builder.Services.AddAuthorization();

var app = builder.Build();

app.UseStaticFiles();

app.UseRouting();

app.UseAuthentication();

app.UseAuthorization();
```

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");  
app.Run();
```

HomeController.cs

```
using Microsoft.AspNetCore.Mvc;  
using Microsoft.AspNetCore.Authorization;  
using Microsoft.AspNetCore.Authentication;  
using Microsoft.AspNetCore.Authentication.Cookies;  
using System.Security.Claims;  
namespace AuthDemo.Controllers  
{  
    public class HomeController : Controller  
    {  
        public IActionResult Index()  
        {  
            return View();  
        }  
        [HttpPost]  
        public async Task<IActionResult> Index(string username, string password)  
        {  
            if (username == "admin" && password == "123")  
            {  
                var claims = new List<Claim>  
                {  
                    new Claim(ClaimTypes.Name, username)  
                };  
                var identity = new ClaimsIdentity(claims,  
                    CookieAuthenticationDefaults.AuthenticationScheme);  
                await HttpContext.SignInAsync(  
                    CookieAuthenticationDefaults.AuthenticationScheme,
```

```
        new ClaimsPrincipal(identity));
        return RedirectToAction("Secure");
    }
    ViewBag.Message = "Invalid Username or Password";
    return View();
}
[Authorize]
public IActionResult Secure()
{
    return Content("Welcome! Login Successful.");
}
public async Task<IActionResult> Logout()
{
    await HttpContext.SignOutAsync(
        CookieAuthenticationDefaults.AuthenticationScheme);
    return RedirectToAction("Index");
}
}
```

Views/Home/Index.cshtml

```
<h2>Login</h2>
<form method="post">
    Username :
    <input type="text" name="username" /><br /><br />
    Password :
    <input type="password" name="password" /><br /><br />
    <input type="submit" value="Login" />
</form>
<p style="color:red">
    @ViewBag.Message
</p>
```

Output:

Login

Username :

Password :

Welcome! Login Successful.